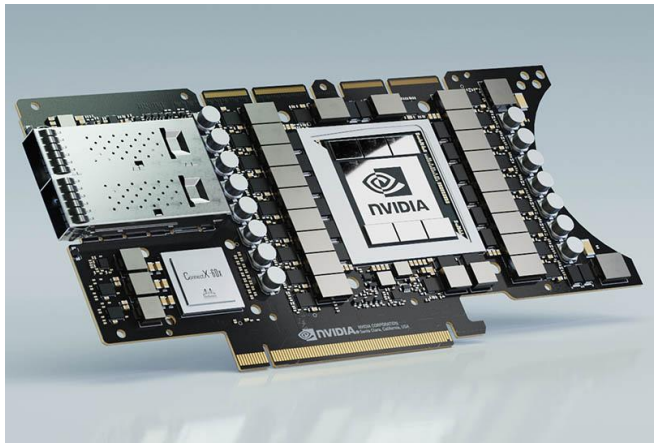
Project 4
Memory & IO Management

Peripheral devices

Graphics Processing Unit

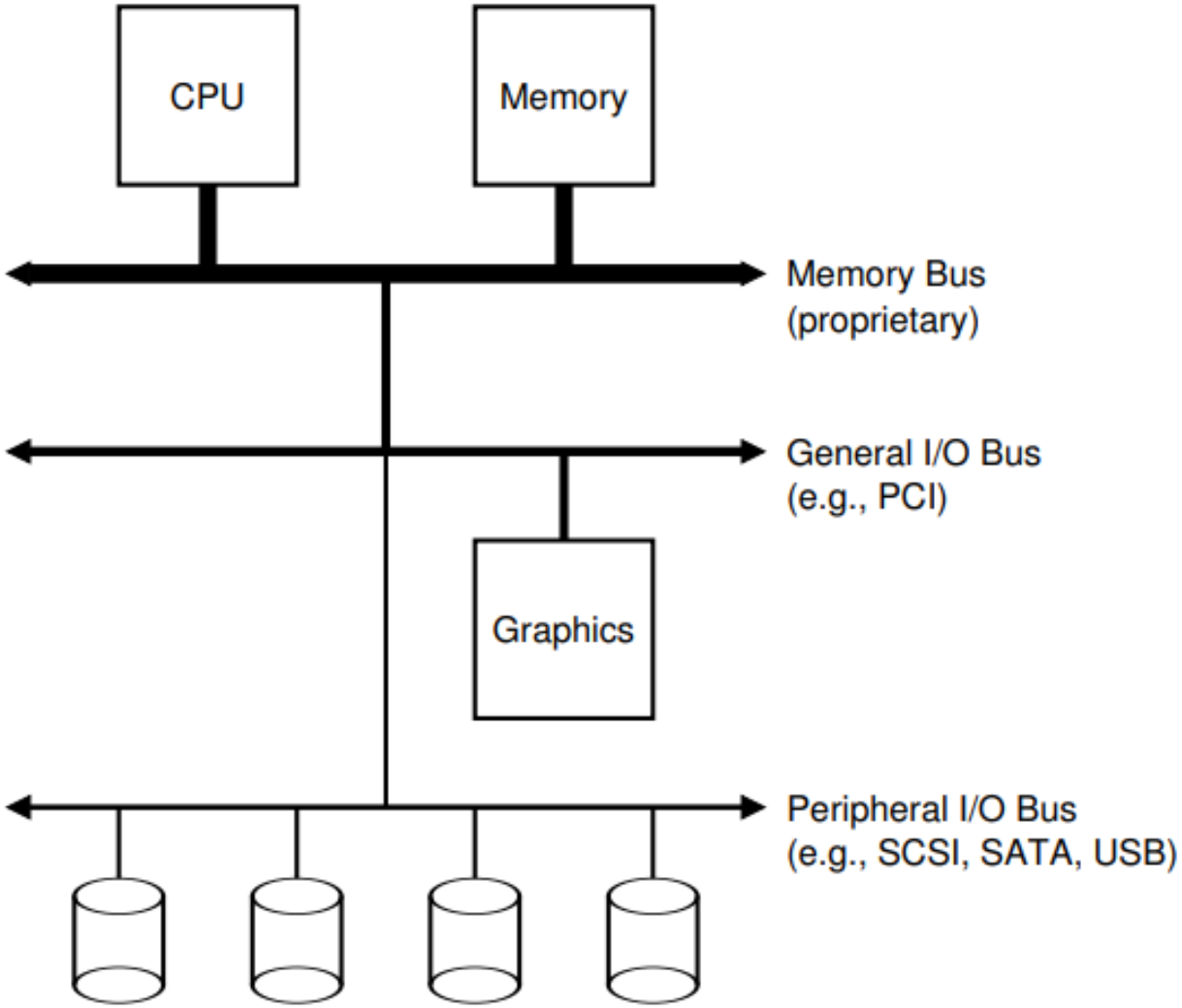
Solid-state Drive



Network Interface Card

Peripheral devices

Generally, devices are connected to the system via **bus**, e.g, PCI.



Read more at: https://pages.cs.wisc.edu/~remzi/OSTEP/file-devices.pdf

Peripheral devices

How to interact with devices?

Memory-mapped IO

```
99:00.0 3D controller: NVIDIA Corporation GA100 [A100 SXM4 40GB] (rev a1)
        Subsystem: NVIDIA Corporation GA100 [A100 SXM4 40GB]
        Physical Slot: 11
        Control: I/O- Mem+ BusMaster+ SpecCycle- MemWINV- VGASnoop- ParErr- Stepping- SERR+ FastB2B- DisINTx+
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR- <PERR- INTx-
        Latency: 0
        Interrupt: pin A routed to IRQ 16
        NUMA node: 1
        Region 0: Memory at d2000000 (32-bit, non-prefetchable) [size=16M]
        Region 1: Memory at 400000000000 (64-bit, prefetchable) [size=64G]
        Region 3: Memory at 401400000000 (64-bit, prefetchable) [size=32M]
        Capabilities: <access denied>
        Kernel driver in use: nvidia
        Kernel modules: nvidiafb, nouveau, nvidia_drm, nvidia
```

36.6 Methods Of Device Interaction

“……With this approach, the hardware makes device registers available as if they were memory locations. To access a particular register, the OS issues a load (to read) or store (to write) the address; the hardware then routes the load/store to the device instead of main memory. ”

Read more at: https://pages.cs.wisc.edu/~remzi/OSTEP/file-devices.pdf

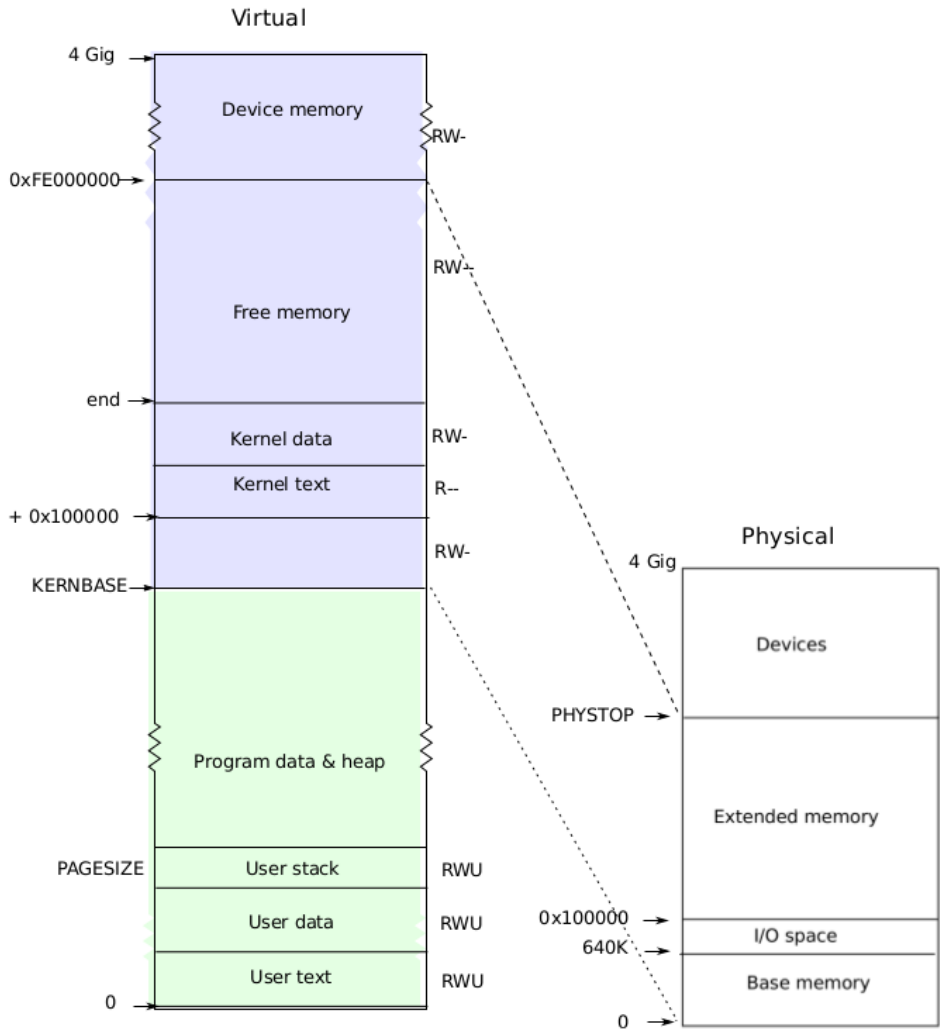
Peripheral devices

Device Memory in xv6

0xFE000000 - 0xFFFFFFFF

Read memlayout.h

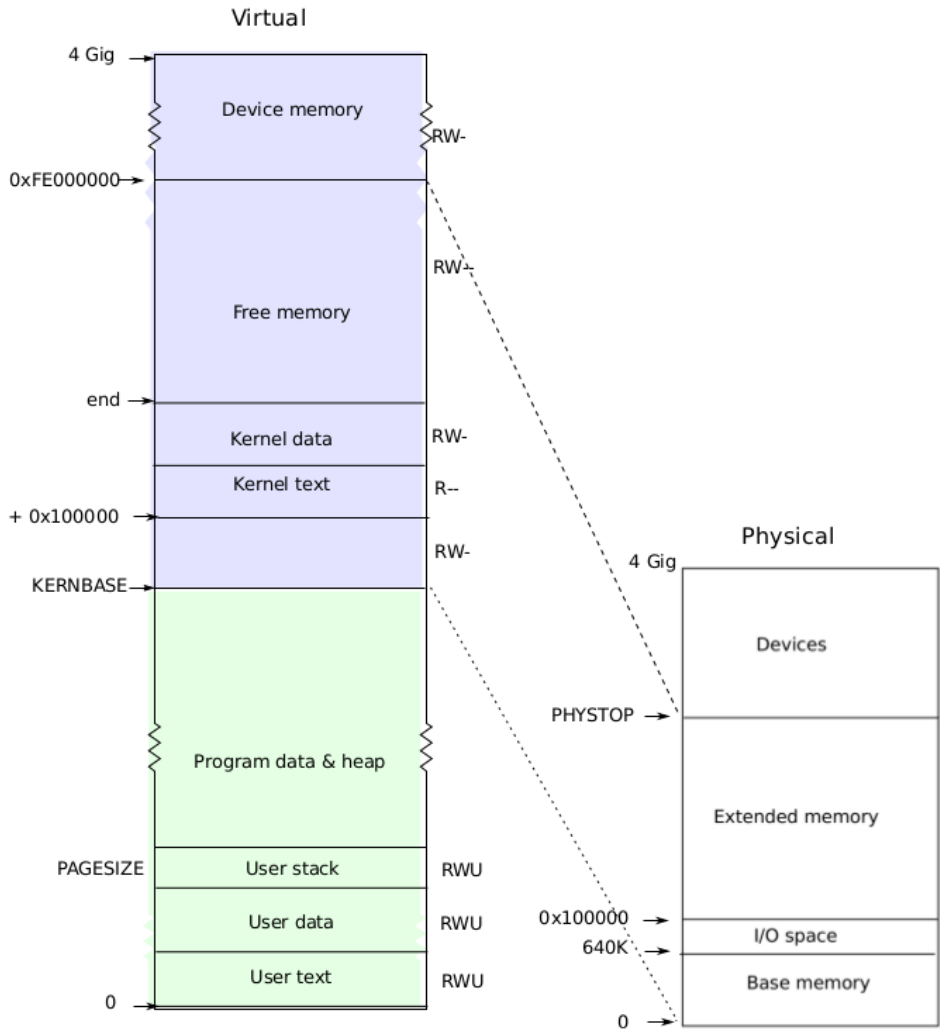
Do you have access to device memory?



Project 4: Two toy devices

A naïve Data Processing Unit
&
A naïve Solid-state Drive

Both expose their control registers. (4KB region)
DPU also exposes its memory.



Task 1 Map Device memory to user-space

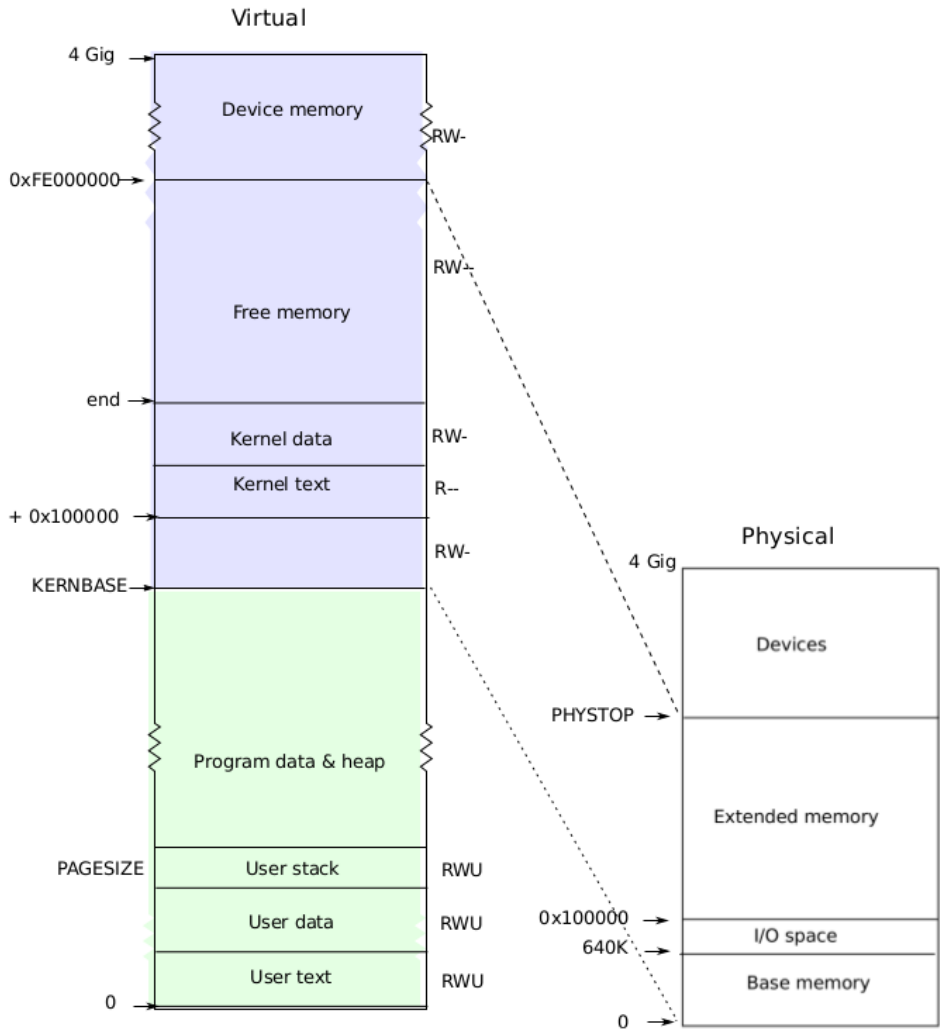
Implement a new syscall

```
18
19  int
20  sys_iomap(void)
21  {
22  // Task 1
23    return 0;
24  }
25
```

Modify deallocuvm()

```
261 int
262 deallocuvm(pde_t *pgdir, uint oldsz, uint newsz)
263 {
```

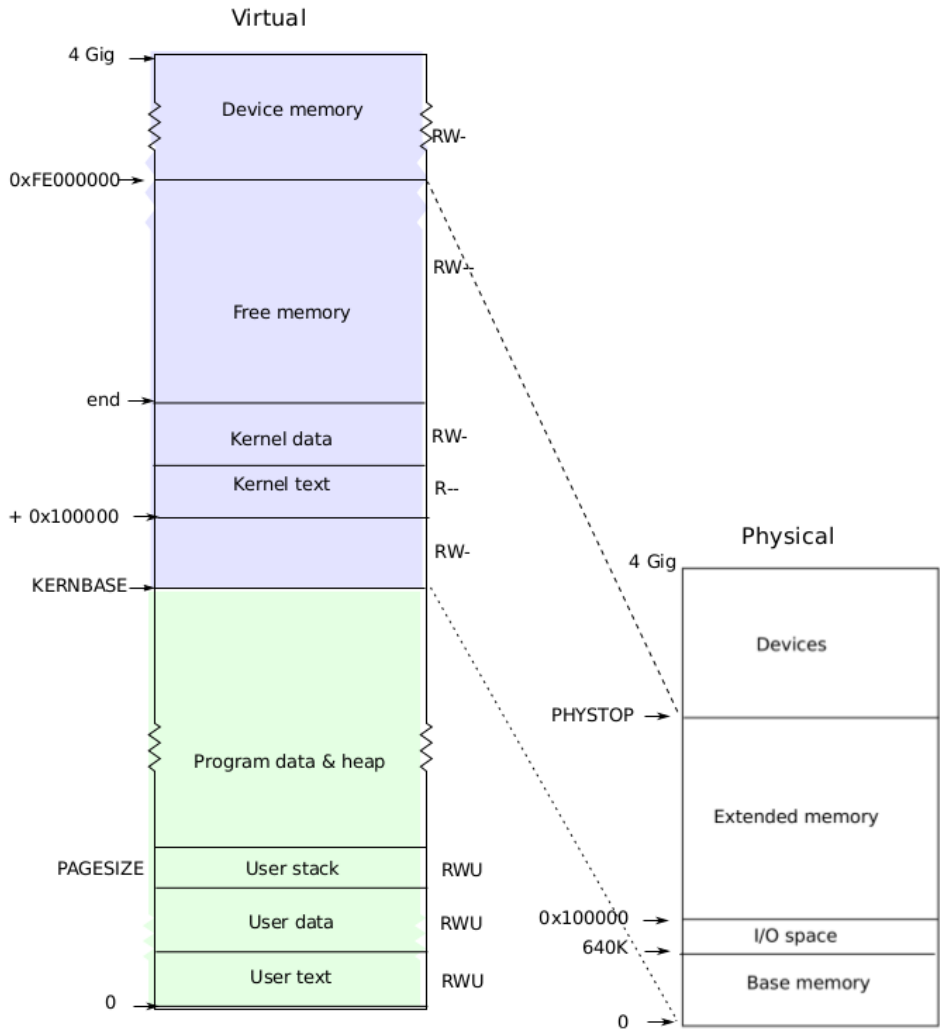
Why?



Task 2 Memory-mapped IO

Test whether your syscall and memory-mapping is correct!

Then you have the read/write access to devices registers and DPU memory!



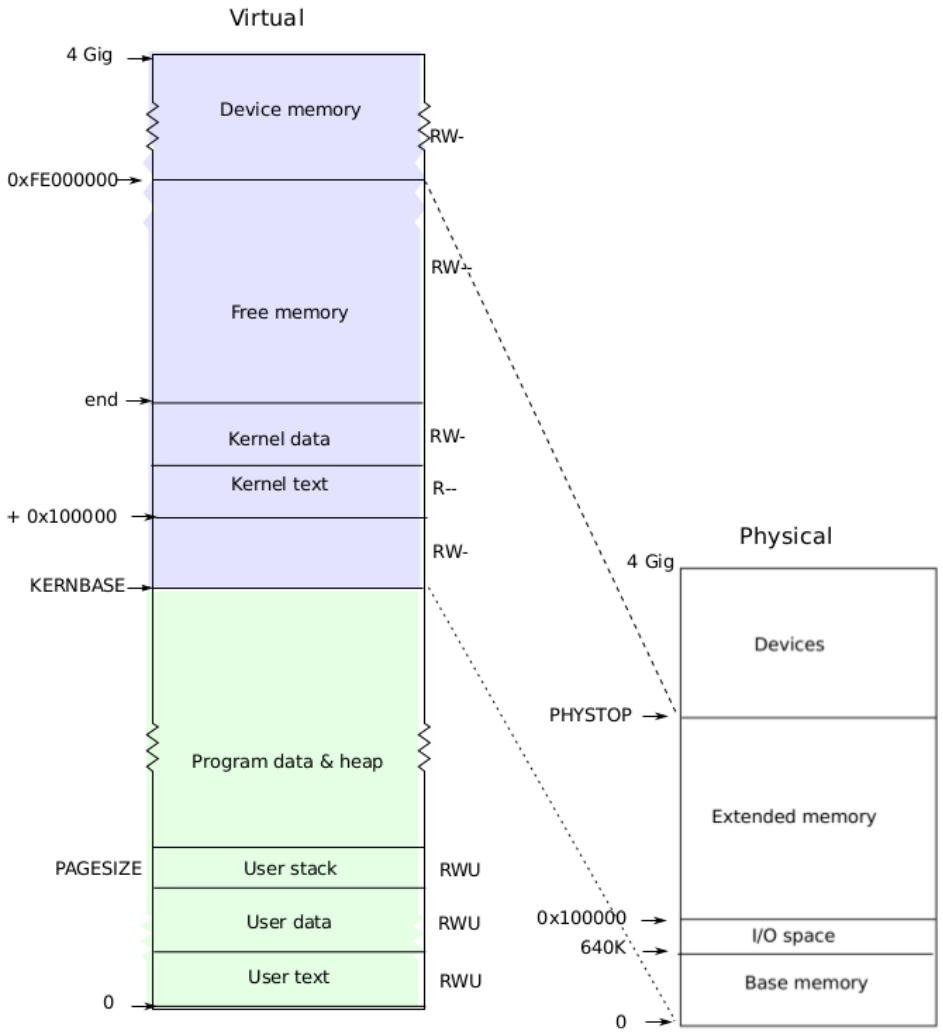
Task 3 Huge Page

32MB huge page memory has already been reserved.

Implement a new syscall

```
26  int
27  sys_hpmap(void)
28  {
29  // Task 3
30    return 0;
31  }
32
```

Why we need to use huge page?
We will see in Task 4!



Task 3 Huge Page

```
26  int
27  sys_hpmap(void)
28  {
29  // Task 3
30    return 0;
31  }
32
```

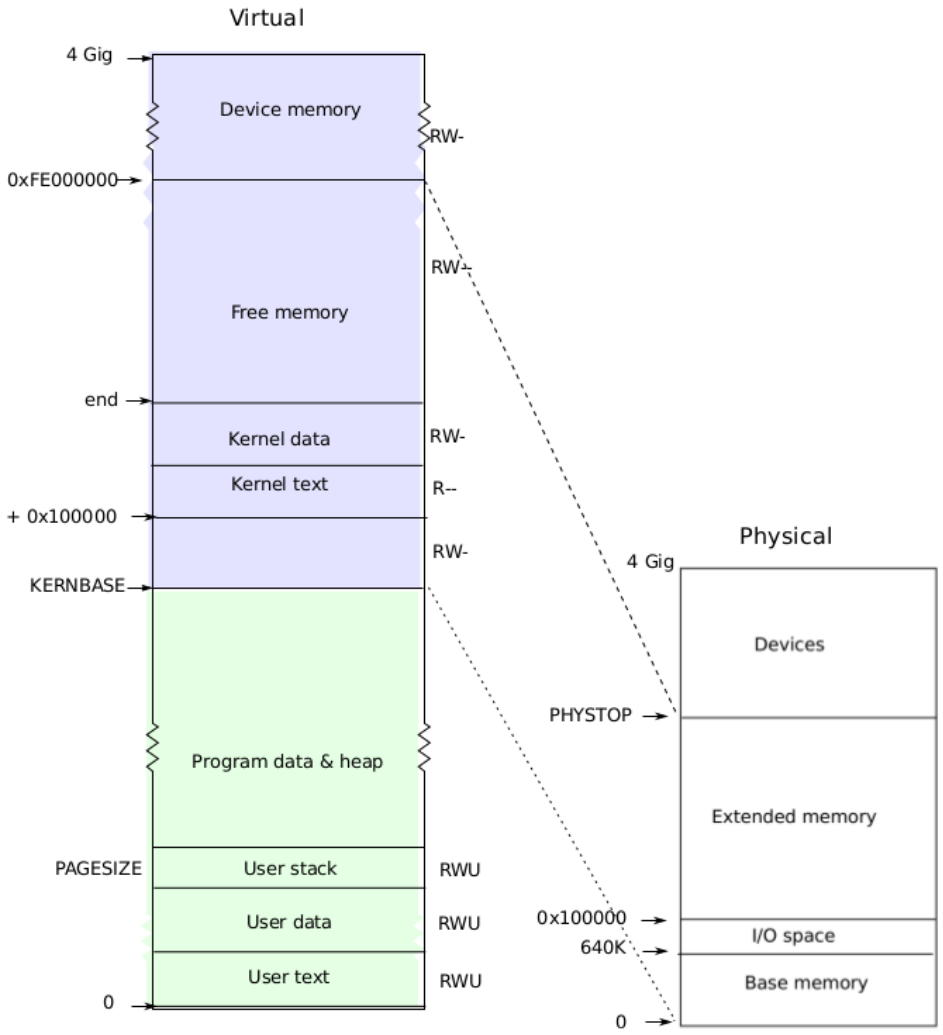
int mappages() works?

```
82  int
83  hpmappages(pde_t *pgdir, void *va, uint size, uint pa, int perm){
84  // Task 3
85    return 0;
86  }
87
```

No. You need a new map function

vm functions to change?

Manage your vm carefully !



Task 4 Direct Memory Access

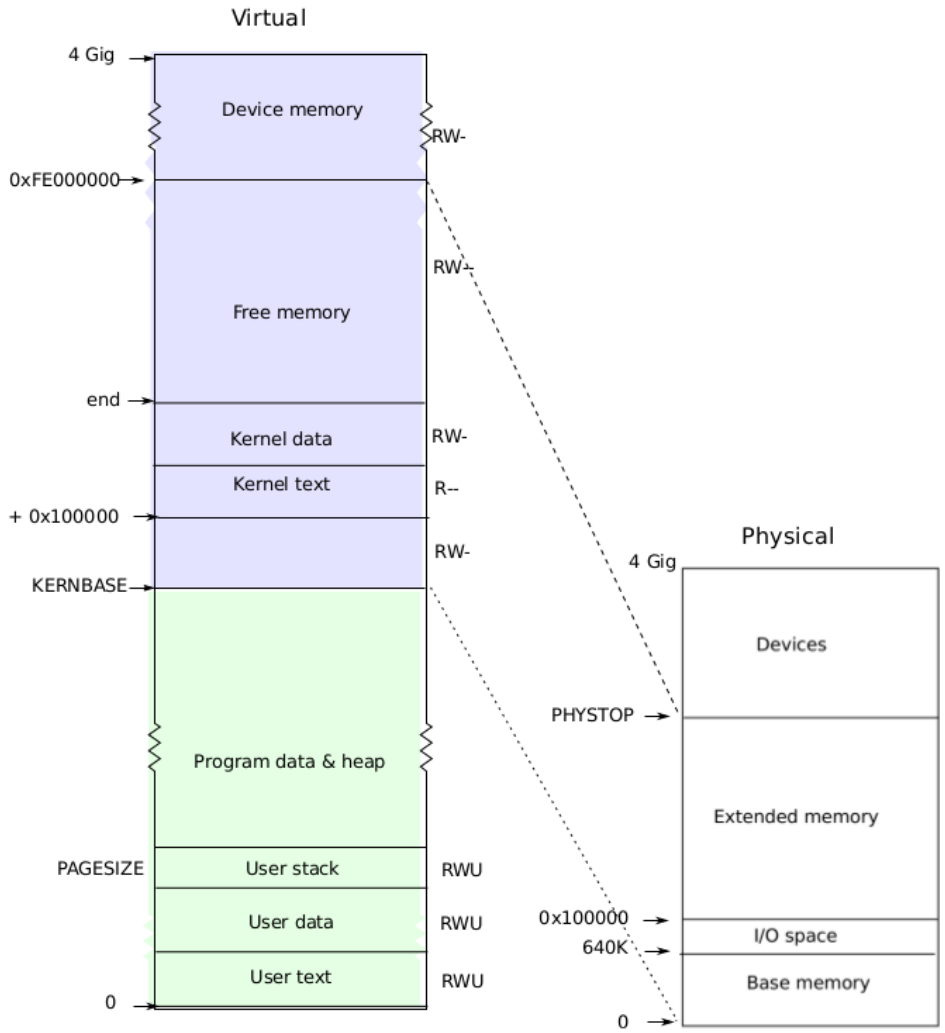
36.6 More Efficient Data Movement With DMA

THE CRUX: HOW TO LOWER PIO OVERHEADS

With PIO, the CPU spends too much time moving data to and from devices by hand. How can we offload this work and thus allow the CPU to be more effectively utilized?

The solution to this problem is something we refer to as **Direct Memory Access (DMA)**. A DMA engine is essentially a very specific device within a system that can orchestrate transfers between devices and main memory without much CPU intervention.

DMA works as follows. To transfer data to the device, for example, the OS would program the DMA engine by telling it where the data lives in memory, how much data to copy, and which device to send it to. At that point, the OS is done with the transfer and can proceed with other work.



DMA is proposed to solve:

- High CPU overheads
- Low throughput

Read more at: https://pages.cs.wisc.edu/~remzi/OSTEP/file-devices.pdf

Task 4 Direct Memory Access

Data in SSD is only accessible through DMA in this project!

device address: SSD space
host buffer address:
What address should it be?
paddr or vaddr?

Why should we use huge page?

Register	Definition
11	Set to 1 to start DMA read. Reset automatically to 0 upon completion.
12	DMA read device address
13	DMA read host buffer address
14	DMA read legnth
15	Host polling address for the DMA read completion
21	Set to 1 to start DMA write. Reset automatically to 0 upon completion.
22	DMA write device address
23	DMA write host buffer address
24	DMA write legnth
25	Host polling address for the DMA write completion

Task 4 Direct Memory Access

For register MMIO region

int* RegRegion

Register 11 is at:

RegRegion[11]

Register	Definition
11	Set to 1 to start DMA read. Reset automatically to 0 upon completion.
12	DMA read device address
13	DMA read host buffer address
14	DMA read legnth
15	Host polling address for the DMA read completion
21	Set to 1 to start DMA write. Reset automatically to 0 upon completion.
22	DMA write device address
23	DMA write host buffer address
24	DMA write legnth
25	Host polling address for the DMA write completion

Task 4 Direct Memory Access

Device polling

36.3 The Canonical Protocol

```
While (STATUS == BUSY)
    ; // wait until device is not busy
Write data to DATA register
Write command to COMMAND register
    (starts the device and executes the command)
While (STATUS == BUSY)
    ; // wait until device is done with your request
```

15	Host polling address for the DMA read completion

What address should it be?
paddr or vaddr?

Polling v.s. Interrupt?

Here we target high-speed devices.

TIP: INTERRUPTS NOT ALWAYS BETTER THAN POLLING

Although interrupts allow for overlap of computation and I/O, they only really make sense for slow devices. Otherwise, the cost of interrupt handling and context switching may outweigh the benefits interrupts provide. There are also cases where a flood of interrupts may overload a system and lead it to livelock [MR96]; in such cases, polling provides more control to the OS in its scheduling and thus is again useful.

Read more at: https://pages.cs.wisc.edu/~remzi/OSTEP/file-devices.pdf

Task 5 Computing Tasks

Put everything together!

Perform two tasks, a hash function & a vector inner product on DPU.

Extension:

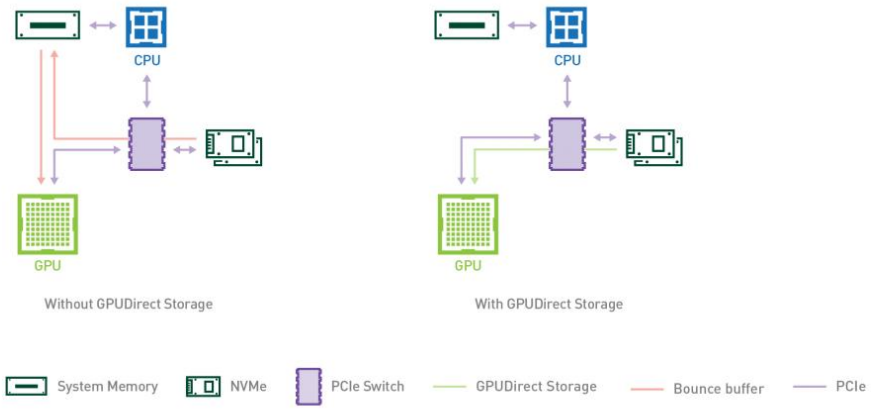
What are we doing (emulating) in this project?

Let’s see some real-world examples.

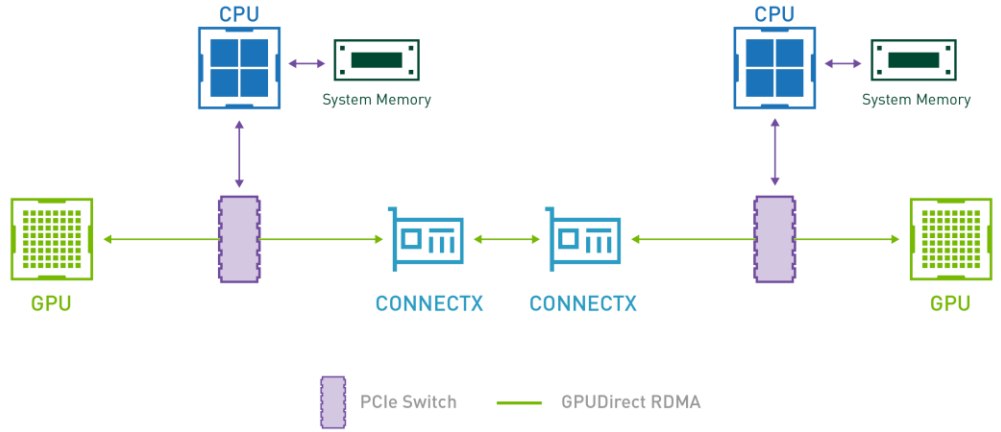
Extension:

What are we doing (emulating) in this project?

Let’s see some real-world examples.



SSD <-> GPU



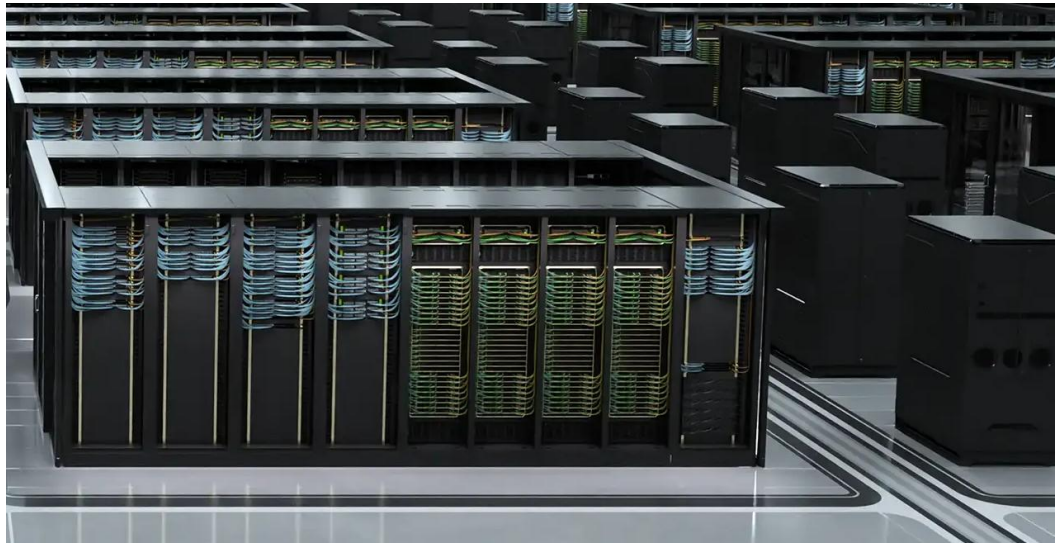
GPU <-> NIC <-> Ethernet <-> NIC <-> GPU

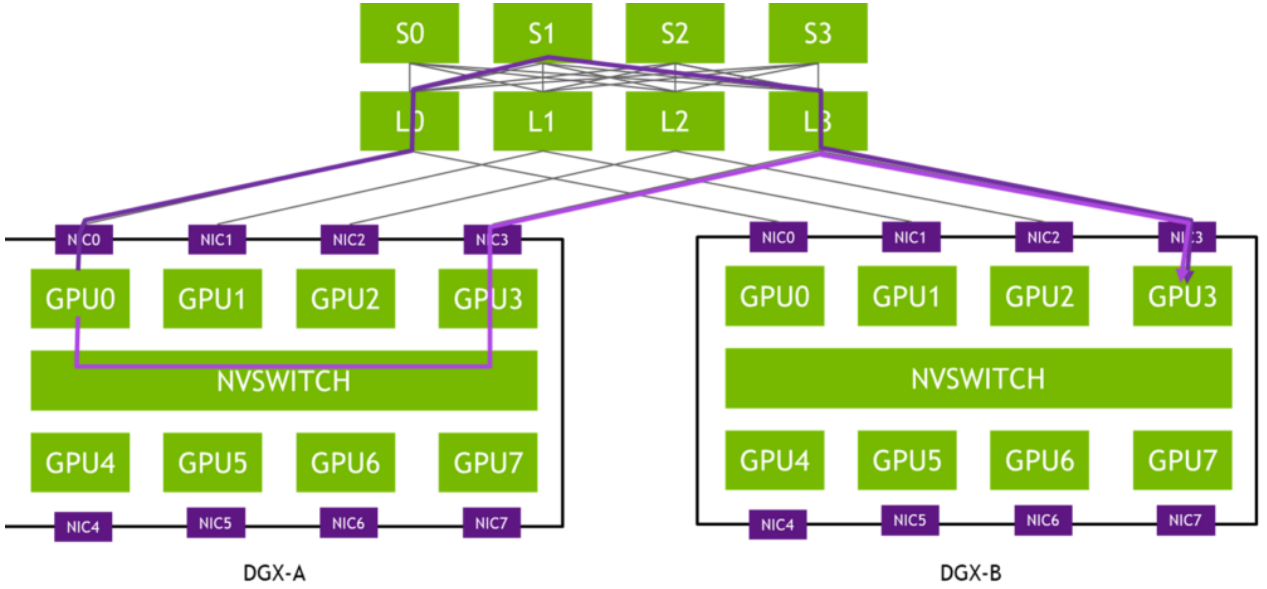
Read more at: https://developer.nvidia.com/gpudirect

Extension:

What are we doing (emulating) in this project?

Let’s see some real-world examples.





GPU training clusters, GPU to GPU, GPU to storage …